DATE : / /
PAGE :

1 From NAND TO NOT (both input same) 1 input 1 NAND



$\overline{A \cdot A} \rightarrow \overline{A}$

Truth Table

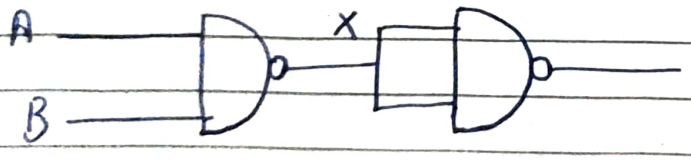
A	NAND A
0	1
1	0

A·A NAND
$\overline{A \cdot A} = \overline{A}$

2. From NAND to AND

$(\overline{A \cdot B})$

2 input 2 Nand

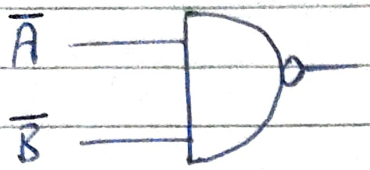


Truth Table

A	B	AND	NOT(AND) = NAND
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

DATE : __/__/____
PAGE : _____

3. From NAND to OR



$\overline{\overline{A} \cdot \overline{B}} = \overline{\overline{A}} + \overline{\overline{B}} = A + B$

#Truth table

A	B	OR	NOR
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

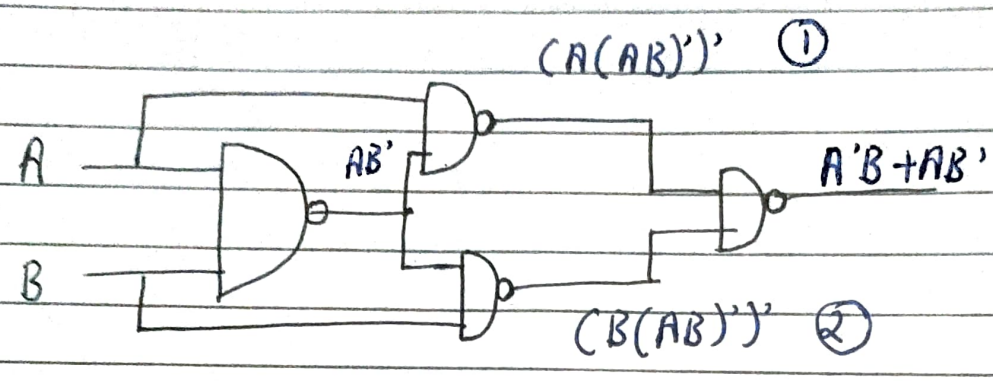
4. From NAND to XOR

XOR diff input → output = 1
same input → output = 0

Truth Table

A	B	output
0	0	0
0	1	1
1	0	1
1	1	0

DATE : __/__/____
PAGE : _____



for eq ① $\overline{(A \cdot (\overline{A \cdot B}))}$

$= \overline{A \cdot \overline{A} + A \cdot \overline{B}}$

$= \overline{A \cdot \overline{B}}$

$= \overline{A} + \overline{\overline{B}} = \overline{A} + B$

for ② $\overline{B \cdot (\overline{B \cdot A})}$

$= \overline{B \cdot \overline{B} + B \cdot \overline{A}}$

$= \overline{B \cdot \overline{A}}$

$= \overline{B} + \overline{\overline{A}} = \overline{B} + A$

$= (\overline{A} + B) \cdot (\overline{B} + A)$

$= \overline{A} \cdot \overline{B} + \overline{A} \cdot A + \overline{B} B + B \cdot A$

$= \overline{A} \cdot \overline{B} + B \cdot A$

$= \overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{B}} + \overline{\overline{A}}$

$= (A + B) + (\overline{B} + \overline{A})$

$= A\overline{B} + A \cdot \overline{A} + B + \overline{B} + B\overline{A}$

$= A \cdot \overline{B} + B \cdot \overline{A}$

HDL --LBSt Project 1 And

```
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And.hdl
/**
 * And gate:
 * if (a and b) out = 1, else out = 0
 */
CHIP And {
    IN a, b;
    OUT out;

    PARTS:
   Nand(a=a, b=b, out=nandOut);
   Nand(a=nandOut, b=nandOut, out=out);
}
```

Chip And — Eval Reset Clock: 0

Input pins

a 1

b 1

Output pins

out 1

Internal pins

nandOut 0

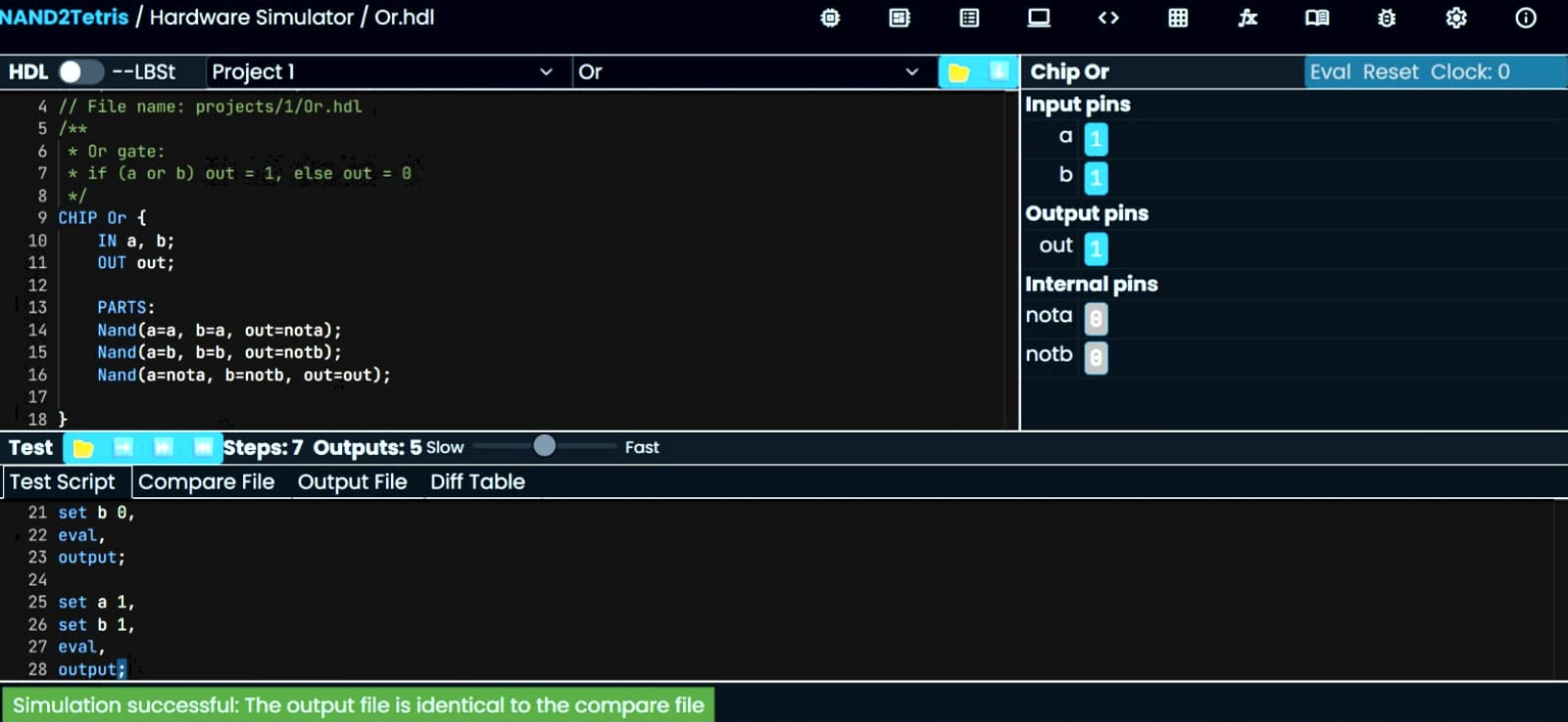
Test — Steps: 8 Outputs: 5 Slow Fast

Test Script | Compare File | Output File | Diff Table

```
set b 0,
eval,
output;

set a 1,
set b 1,
eval,
output;
```

Simulation successful: The output file is identical to the compare file


NAND2Tetris / Hardware Simulator / Or.hdl
HDL --LBSt
Project 1
Or
4 // File name: projects/1/Or.hdl
5 /**
6 * Or gate:
7 * if (a or b) out = 1, else out = 0
8 */
9 CHIP Or {
10 IN a, b;
11 OUT out;
12
13 PARTS:
14 Nand(a=a, b=a, out=nota);
15 Nand(a=b, b=b, out=notb);
16 Nand(a=nota, b=notb, out=out);
17
18 }
Chip Or
Eval Reset Clock: 0
Input pins
a 1
b 1
Output pins
out 1
Internal pins
nota 0
notb 0
Test
Steps: 7 Outputs: 5 Slow
Fast
Test Script
Compare File
Output File
Diff Table
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
Simulation successful: The output file is identical to the compare file

HDL --LBSt Project 1 Xor

```
/**
 * Exclusive-or gate:
 * if ((a and Not(b)) or (Not(a) and b)) out = 1, else out = 0
 */
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a,b=b,out=x);
    Nand(a=a,b=x,out=y);
    Nand(a=b,b=x,out=z);
    Nand(a=y,b=z,out=out);

}
```

Chip Xor Eval Reset Clock: 0

Input pins
- a 1
- b 1

Output pins
- out 0

Internal pins
- x 0
- y 1
- z 1

Test Steps: 6 Outputs: 5 Slow Fast

Test Script | Compare File | Output File | Diff Table

```
set b 0,
eval,
output;

set a 1,
set b 1,
eval,
output;
```

Simulation successful: The output file is identical to the compare file

HDL --LBSt Project 1 Mux

```
/**
 * Multiplexor:
 * if (sel = 0) out = a, else out = b
 */
CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
    Not(in=sel, out=nsel);
    And(a=a, b=nsel, out=a1);
    And(a=b, b=sel, out=b1);
    Or(a=a1, b=b1, out=out);

}
```

Chip Mux Eval Reset Clock: 0

Input pins
a 1
b 1
sel 1

Output pins
out 1

Internal pins
nsel 0
a1 0

Test Steps: 11 Outputs: 9 Slow Fast

Test Script | Run | pare File | Output File | Diff Table

```
set b 1,
set sel 0,
eval,
output;

set sel 1,
eval,
output;
```

Simulation successful: The output file is identical to the compare file

HDL --LBSt Project 1 DMux

```
/**
 * Demultiplexor:
 * [a, b] = [in, 0] if sel = 0
 *          [0, in] if sel = 1
 */
CHIP DMux {
    IN in, sel;
    OUT a, b;

    PARTS:
    Not(in=sel,out=x);
    And(a=in,b=x,out=a);
    And(a=in ,b=sel,out=b);

}
```

Chip DMux Eval Reset Clock: 0

Input pins

in 1

sel 1

Output pins

a 0

b 1

Internal pins

x 0

Test Steps: 7 Outputs: 5 Slow Fast

Test Script | Compare File | Output File | Diff Table

```
set in 1,
set sel 0,
eval,
output;

set sel 1,
eval,
output;
```

Simulation successful: The output file is identical to the compare file

HDL --LBSt | Project 2 | HalfAdder

```
/**
 * Computes the sum of two bits.
 */
CHIP HalfAdder {
    IN a, b;    // 1-bit inputs
    OUT sum,    // Right bit of a + b
        carry;  // Left bit of a + b

    PARTS:
    Xor(a=a , b=b , out=sum );
    And(a=a , b=b , out=carry);


    //// Replace this comment with your code.
}
```

Chip HalfAdder | Eval Reset Clock: 0

Input pins

a 1

b 1

Output pins

sum 0

carry 1

Test | Steps: 7 Outputs: 5 Slow Fast

Test Script | Compare File | Output File | Diff Table

```
set b 0,
eval,
output;

set a 1,
set b 1,
eval,
output;
```

Simulation successful: The output file is identical to the compare file

HDL --LBSt Project 2 FullAdder

```
/**
 * Computes the sum of three bits.
 */
CHIP FullAdder {
    IN a, b, c;  // 1-bit inputs
    OUT sum,     // Right bit of a + b + c
        carry;   // Left bit of a + b + c

    PARTS:
    HalfAdder(a=a , b=b , sum=s1 , carry=c1 );
    HalfAdder(a=s1 , b=c , sum=sum , carry=c2 );
    Or(a=c1 , b=c2 , out=carry );

    //// Replace this comment with your code.
}
```

Chip FullAdder Eval Reset Clock: 0

Input pins
a 1
b 1
c 1

Output pins
sum 1
carry 1

Internal pins
s1 0

Test Steps: 11 Outputs: 9 Slow Fast

Test Script | Compare File | Output File | Diff Table

```
set b 1,
set c 0,
eval,
output;

set c 1,
eval,
output;
```

Simulation successful: The output file is identical to the compare file

HDL --LBSt Project 2 Add16

```
/**
 */
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c1);

    FullAdder(a=a[1], b=b[1], c=c1, sum=out[1], carry=c2);
    FullAdder(a=a[2], b=b[2], c=c2, sum=out[2], carry=c3);
    FullAdder(a=a[3], b=b[3], c=c3, sum=out[3], carry=c4);
    FullAdder(a=a[4], b=b[4], c=c4, sum=out[4], carry=c5);
    FullAdder(a=a[5], b=b[5], c=c5, sum=out[5], carry=c6);
    FullAdder(a=a[6], b=b[6], c=c6, sum=out[6], carry=c7);
```

Chip Add16 Eval Reset Clock: 0

Input pins

a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 dec

b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0 dec

Output pins

out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 dec

Internal pins

c1 0

c2 0

c3 1

Test **Steps: 9 Outputs: 7** Slow Fast

Test Script | Compare File | Output File | Diff Table

```
set b %B0000111111110000,
eval,
output;

set a %B0001001000110100,
set b %B1001100001110110,
eval,
output;
```

Simulation successful: The output file is identical to the compare file

HDL --LBSt Project 2 ALU

```
28
29 CHIP ALU {
30     IN
31         x[16], y[16],  // 16-bit inputs
32         zx, // zero the x input?
33         nx, // negate the x input?
34         zy, // zero the y input?
35         ny, // negate the y input?
36         f,  // compute (out = x + y) or (out = x & y)?
37         no; // negate the out output?
38     OUT
39         out[16], // 16-bit output
40         zr,      // if (out == 0) equals 1, else 0
41         ng;      // if (out < 0)  equals 1, else 0
42
43     PARTS:
```

Chip ALU Eval Reset Clock: 0

Input pins

x	0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1	dec
y	0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1	dec
zx	0	
nx	1	
zy	0	
ny	1	
f	0	

Test Steps: 41 Outputs: 37 Slow Fast

Test Script | Compare File | Output File | Diff Table

```
367 set zx 0,
368 set nx 1,
369 set zy 0,
370 set ny 1,
371 set f  0,
372 set no 1,
373 eval,
374 output;
```

Simulation successful: The output file is identical to the compare file

DATE : / /
PAGE :

ALU Functions Table

zx	nx	zy	ny	f	no	function
1	0	1	0	1	0	0
1	1	1	1	1	1	+1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	1	1	1	1	1	-x
1	1	0	1	1	1	-y
0	1	1	1	1	0	x+1
1	1	0	1	1	0	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x|y

DATE : / /
PAGE :

ALU Functions Table

zx	nx	zy	ny	f	no	fuction
1	0	1	0	1	0	0
1	1	1	1	1	1	+1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	!x
1	1	0	0	0	1	!y
0	1	1	1	1	1	-x
1	1	0	1	1	1	-y
0	1	1	1	1	0	x+1
1	1	0	1	1	0	y+1
0	0	1	1	1	0	x-1
1	1	0	0	1	0	y-1
0	0	0	0	1	0	x+y
0	1	0	0	1	1	x-y
0	0	0	1	1	1	y-x
0	0	0	0	0	0	x&y
0	1	0	1	0	1	x|y